

MEMORIA + RAZONAMIENTO + APRENDIZAJE (FIN 1 GROK-4.3)

Análisis y Recomendación

Después de revisar las 4 estructuras que propones, te doy un análisis claro y directo:

Evaluación Rápida

Estructura	Claridad	Separación de Memoria	Madurez Conceptual	Facilidad de Mantenimiento	Puntuación
Estructura 1	Media	Buena	Media	Media	6.5
Estructura 2	Baja	Media	Alta (código)	Baja	5.5
Estructura 3	Media-Alta	Excelente	Muy Alta	Media	8.0
Estructura 4	Alta	Muy Buena	Alta	Alta	9.0

Análisis Detallado

Estructura 1 y 2 son las más débiles para tu objetivo:

- La 1 está bien como punto de partida, pero le falta profundidad en cómo se gestiona la memoria a largo plazo.
- La 2 es más un *framework* que una estructura de proyecto. Es demasiado pesada y está pensada más para desarrollo de herramientas que para acumular conocimiento de forma limpia.

Estructura 3 es la más ambiciosa y conceptualmente madura. Tiene muy buena separación entre:

- Memoria
- Razonamiento
- Conocimiento

Sin embargo, puede volverse difícil de mantener con el tiempo por tener tantas carpetas.

Estructura 4 es la que mejor equilibra todo. Es limpia, clara, y tiene una muy buena organización de tipos de memoria (STM, LTM, Episodic), que es clave para lo que quieres lograr.

Recomendación Final

Te recomiendo usar la Estructura 4 como base, pero incorporando algunos elementos fuertes de la Estructura 3.

Estructura Recomendada (Híbrida)

Bash

```
mi-proyecto/
├── .ai/
│   ├── system.md
│   ├── rules.md
│   ├── context.md
│   └── agents.md
├── memory/
│   ├── README.md
│   ├── STM.md
│   ├── LTM.md
│   ├── episodic.md
│   ├── consolidation.md
│   └── retrieval.md
└── # ← Sistema de memoria
    # Cómo funciona todo el sistema
    # Memoria de corto plazo (sesión actual)
    # Memoria de largo plazo (conocimiento acumulado)
    # Memoria episódica (proyectos/decisiones)
    # (de la estructura 3) Cómo consolidar memoria
    # (de la estructura 3) Cómo recuperar información
```

```

├── index.md
├── reasoning/                                # ← Sistema de razonamiento
│   ├── CoT.md
│   ├── factcheck.md
│   └── patterns.md                          # Patrones de razonamiento aprendidos
├── knowledge/                               # ← Conocimiento acumulado
│   ├── architecture.md
│   ├── decisions.md
│   ├── patterns.md
│   └── mistakes.md
├── logs/
│   ├── decisions.md
│   ├── errors.md
│   └── learnings.md
├── memory/                                 # ← Datos reales (legibles + vectoriales)
│   ├── chroma/                             # Base vectorial (ChromaDB)
│   ├── episodic/                           # Episodios en JSON/MD
│   ├── semantic/                           # Hechos y conocimiento
│   └── summaries/
├── .ai_memory/                             # Backups / exports
├── src/
├── .env
└── opencode.json

```

¿Por qué esta combinación?

- Estructura 4 → Claridad y organización general (la mejor base).
- Elementos de Estructura 3 → `consolidation.md`, `retrieval.md`, `knowledge/` y `mistakes.md` (estos son muy importantes para que el sistema realmente "aprenda").
- Separa claramente Memoria vs Razonamiento vs Conocimiento (esto es clave para escalabilidad).

Consejo Adicional

Si quieres que el sistema realmente mejore con el tiempo, prioriza tener bien definidos estos dos archivos:

1. `memory/consolidation.md` → Reglas de cómo y cuándo consolidar información.
2. `memory/retrieval.md` → Cómo debe buscar y usar la memoria el agente.

¿Quieres que te ayude a definir el contenido de estos archivos clave (consolidation.md, retrieval.md y [LTM.md](#))?

✓ Respuesta: Voy a generarte una implementación completa y funcional del sistema de memoria para que lo puedas probar en OpenCode.

Te voy a entregar:

1. Estructura de carpetas final (la híbrida recomendada)
2. Código principal del sistema de memoria
3. Servidor MCP para que OpenCode pueda usarlo
4. Juego de pruebas (tests) para validar que funciona correctamente

1. Estructura Final del Proyecto

Bash

```
mi-proyecto/
```

```
├── .ai/
```

```
│   ├── memory/
```

```
│       ├── README.md
```

```
│       ├── STM.md
```

```
│       ├── LTM.md
```

```
│       ├── episodic.md
```

```
│       ├── consolidation.md
```

```
│       └── retrieval.md
```

```
│   └── reasoning/
```

```
│       ├── CoT.md
```

```
│       └── factcheck.md
```

```
└── memory/
```

```
│   ├── chroma/                # Base de datos vectorial
```

```
│   ├── episodic/              # Memorias episódicas
```

```
|   └─ semantic/                # Lecciones y patrones
|
|─ ai_architect/
|
|   └─ core/
|
|       └─ __init__.py
|
|       └─ memory.py            # ← Núcleo principal
|
|       └─ mcp_server.py        # ← Servidor MCP para OpenCode
|
|   └─ config/
|
|       └─ settings.py
|
|─ tests/
|
|   └─ test_memory.py
|
|   └─ test_consolidation.py
|
|─ requirements.txt
|
|─ memory_config.py
|─ main.py
```

2. Código Principal

requirements.txt

txt

chromadb>=0.4.22

sentence-transformers>=2.2.2

pydantic>=2.0

python-dotenv>=1.0

ai_architect/core/memory.py

Python

```
import chromadb
```

```
from chromadb.utils import embedding_functions
```

```
from datetime import datetime
```

```

import json

import os

from typing import List, Dict, Optional


class MemorySystem:

    def __init__(self, persist_directory: str = "./memory/chroma"):

        self.client = chromadb.PersistentClient(path=persist_directory)

        self.embedding_fn = embedding_functions.SentenceTransformerEmbeddingFunction(

            model_name="all-MiniLM-L6-v2"

        )

        self.collection = self.client.get_or_create_collection(

            name="long_term_memory",

            embedding_function=self.embedding_fn

        )

        self.episodic_path = "./memory/episodic"

        self.semantic_path = "./memory/semantic"

        os.makedirs(self.episodic_path, exist_ok=True)

        os.makedirs(self.semantic_path, exist_ok=True)

    def store_episode(self, project: str, decision: str, context: str, outcome: str, tags:
List[str] = None):

        """Guarda una memoria episódica"""

        episode = {

            "timestamp": datetime.now().isoformat(),

            "project": project,

```

```

        "decision": decision,

        "context": context,

        "outcome": outcome,

        "tags": tags or []

    }

    filename = f"{datetime.now().strftime('%Y%m%d_%H%M%S')}_{project}.json"

    path = os.path.join(self.episodic_path, filename)

    with open(path, "w", encoding="utf-8") as f:

        json.dump(episode, f, indent=2, ensure_ascii=False)

    # También lo guarda en ChromaDB

    self.collection.add(

        documents=[f"{decision}. Contexto: {context}. Resultado: {outcome}"],

        metadatas=[{"type": "episode", "project": project, "tags": str(tags)}],

        ids=[filename]

    )

    return filename


def store_lesson(self, lesson: str, category: str = "general", confidence: float = 0.8):

    """Guarda una lección o patrón aprendido"""

    doc_id = f"lesson_{datetime.now().strftime('%Y%m%d_%H%M%S')}"

    self.collection.add(

        documents=[lesson],

```

```

        metadatas=[{
            "type": "lesson",
            "category": category,
            "confidence": confidence,
            "timestamp": datetime.now().isoformat()
        }],
        ids=[doc_id]
    )

    return doc_id

def retrieve_relevant(self, query: str, n_results: int = 5) -> List[Dict]:
    """Recupera memorias relevantes"""

    results = self.collection.query(
        query_texts=[query],
        n_results=n_results,
        include=["documents", "metadatas", "distances"]
    )

    memories = []

    for i in range(len(results['documents'][0])):
        memories.append({
            "content": results['documents'][0][i],
            "metadata": results['metadatas'][0][i],
            "distance": results['distances'][0][i]
        })

    return memories

```



```
def consolidate(self):  
  
    """Consolida memorias (versión básica)"""  
  
    print("Consolidación ejecutada (versión básica)")  
    # Aquí puedes agregar lógica más avanzada después
```

3. Servidor MCP para OpenCode

ai_architect/core/mcp_server.py

Python

```
from mcp.server import Server  
  
from mcp.server.stdio import stdio_server  
  
from ai_architect.core.memory import MemorySystem  
  
  
server = Server("ai-memory")  
  
memory = MemorySystem()  
  
  
@server.tool()  
  
async def store_episode(project: str, decision: str, context: str, outcome: str, tags: list  
= None):  
  
    return memory.store_episode(project, decision, context, outcome, tags)  
  
  
@server.tool()  
  
async def store_lesson(lesson: str, category: str = "general"):  
  
    return memory.store_lesson(lesson, category)  
  
  
@server.tool()  
  
async def retrieve_memory(query: str, n_results: int = 5):  
  
    return memory.retrieve_relevant(query, n_results)
```

```
if __name__ == "__main__":  
  
    import asyncio  
    asyncio.run(stdio_server(server))
```

4. Juego de Pruebas (Tests)

Crea la carpeta `tests/` y agrega:

`tests/test_memory.py`

Python

```
import pytest  
  
from ai_architect.core.memory import MemorySystem  
  
import os  
  
import shutil  
  
  
@pytest.fixture  
def memory_system():  
  
    # Usamos una carpeta temporal para tests  
  
    test_dir = "./memory/test_chroma"  
  
    if os.path.exists(test_dir):  
  
        shutil.rmtree(test_dir)  
  
  
    mem = MemorySystem(persist_directory=test_dir)  
  
    yield mem  
  
    shutil.rmtree(test_dir, ignore_errors=True)  
  
  
def test_store_episode(memory_system):
```

```

result = memory_system.store_episode(

    project="test-project",

    decision="Usar arquitectura modular",

    context="Proyecto grande con múltiples agentes",

    outcome="Mejor mantenibilidad",

    tags=["arquitectura", "buena-practica"]

)

assert result is not None


def test_store_lesson(memory_system):

    result = memory_system.store_lesson(

        lesson="Siempre separar memoria de corto y largo plazo",

        category="memoria"

    )

    assert result is not None


def test_retrieve_relevant(memory_system):

    memory_system.store_lesson("Usar ChromaDB para memoria persistente")

    results = memory_system.retrieve_relevant("memoria persistente", n_results=3)

    assert len(results) > 0

    assert "memoria" in results[0]["content"].lower()


def test_consolidation_runs(memory_system):

    memory_system.consolidate() # Solo verifica que no lance error

```

5. Cómo Ejecutarlo

1. Instala dependencias:
2. Bash
3. `pip install -r requirements.txt`
4. Ejecuta los tests:
5. Bash
6. `pytest tests/ -v`
7. Para usar con OpenCode, agrega el MCP en tu `opencode.json`:

JSON

```
{  
  
  "mcpServers": {  
  
    "ai-memory": {  
  
      "command": "python",  
  
      "args": ["ai_architect/core/mcp_server.py"]  
  
    }  
  
  }  
  
}
```

¿Quieres que ahora te genere también los archivos de configuración de memoria (como `consolidation.md` y `retrieval.md`) con reglas concretas? O prefieres que primero probemos esta base y luego la mejoramos.